

Computing Methods for Physics – 16 January 2023

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 12:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

1. Design and write a class `Particle` with members representing mass (m), position ($\vec{x}$, with 3 components), and velocity ($\vec{v}$, also with 3 components).
2. Design and write a class `Ensemble` to represent an ensemble of N instances of `Particle`. This class must have at least a constructor that takes N as an argument, and draws the $\{m, \vec{x}, \vec{v}\}_i, i \in \{1, \dots, N\}$, values randomly from distributions of your choice. This class must also store a few global properties: the total mass of the ensemble, its center of mass, and its temperature, defined as

$$T = \frac{2}{3} \frac{\langle K \rangle}{k_B},$$

where $\langle K \rangle$ is the average kinetic energy and k_B is the Boltzmann constant.

3. Allow the user to customize the way the set of N particles is assembled by picking specific uniform/Gaussian distributions from which the $\{m, \vec{x}, \vec{v}\}_i, i \in \{1, \dots, N\}$, values are drawn.
4. Write an application `app.cpp` that creates an instance of `Ensemble` with 10^3 particles, drawing masses, coordinates, and velocity components from specific Gaussian distributions; plots the distributions of these values with ROOT and fits them to show that the original distributions are recovered; and, finally, stores the instance of `Ensemble` to file with ROOT.

Part 2 – Python

Consider the parabolic motion of a projectile of mass m , as we did during the lectures. Expand the methods we saw to handle the situation in which at some moment in time specified by the user the projectile — due to an internal explosion for example — breaks into N multiple parts, with N also specified by the user.

Assume that the projectile parts have equal mass ($m_i = m/N$). Further, assume that, in the center of mass of the projectile, right after the explosion, one of these parts always moves upwards vertically, (i.e., in the opposite direction of $\vec{g}$) and the other velocities are evenly spaced around 2π (see the following scheme for some examples). Note that problem remains a $2D$ problem with these choices.

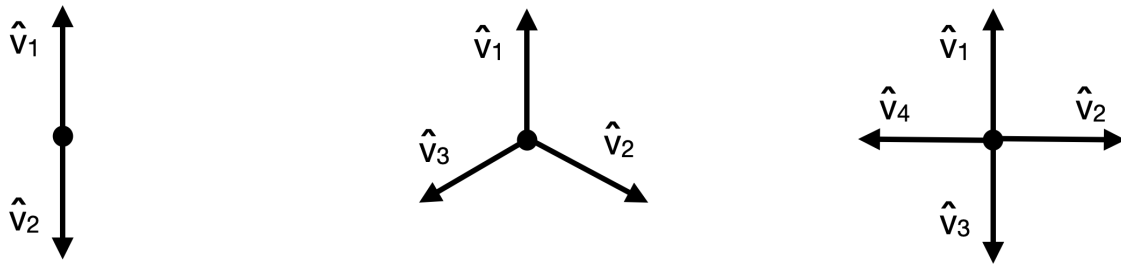


Figure 1: Velocity vector directions when the projectile breaks in 2, 3, and 4 parts.

All objects, including the projectile, can be treated as pointlike particles. The initial speeds of the projectile parts right after the explosion are given by

$$|v_i| = \sqrt{\frac{2E}{m}}$$

in the center of mass of the projectile. E is an energy value specified by the user which quantifies the entity of the explosion.

Use a Python notebook or Python scripts to complete the following tasks.

1. Simulate the kinematics of the projectile and its parts by solving the equations of motion in the inertial reference frame used in class.
2. Plot the kinematic evolution of the system (animating may be time consuming, so it is not mandatory).
3. Store the results of a simulation in a format of your choice.